

Informe de Laboratorio 05

Tema: Arbol AVL

Nota

Estudiante	Escuela	Asignatura
Sebastian Arley Chirinos Negrón schirinosne@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Estructura de datos y Algoritmos Semestre: I Código: 1702124

Laboratorio	Tema	Duración
05	Arbol AVL	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	19 Junio 2023	26 Junio 2023

1. Tarea

- **Introducción** En este informe, se presenta la implementación de Árboles AVL, una estructura de datos balanceada, que se utiliza para almacenar elementos en forma de árbol binario. El objetivo principal de los Árboles AVL es mantener el balance de altura en los subárboles izquierdo y derecho, asegurando que la diferencia de alturas sea como máximo 1. Además, se implementarán todas las operaciones básicas, como búsqueda, obtención del valor mínimo y máximo, obtención del padre de un nodo, inserción y eliminación.
- **Implementación** Para implementar el Árbol AVL, se utilizará un lenguaje de programación como Java. La implementación se basará en clases y métodos que representarán los nodos del árbol y realizarán las operaciones requeridas. A continuación, se describirán las operaciones principales que se implementarán:
 - **search()**
Descripción: Este método se utiliza para buscar un elemento específico en el Árbol AVL. Parámetros: El elemento a buscar.
 - **getMin()**
Descripción: Este método devuelve el valor mínimo presente en el Árbol AVL. Parámetros: No requiere parámetros adicionales.
 - **getMax()**
Descripción: Este método devuelve el valor máximo presente en el Árbol AVL. Parámetros: No requiere parámetros adicionales.

- **parent()**

Descripción: Este método devuelve el padre de un nodo dado en el Árbol AVL. Parámetros: El nodo del cual se desea obtener el padre.

- **son()**

Descripción: Este método verifica si dos nodos son hijos entre sí en el Árbol AVL. Parámetros: Los dos nodos que se desean verificar.

- **insert()**

Descripción: Este método permite insertar un nuevo elemento en el Árbol AVL. El árbol se ajusta automáticamente para mantener la propiedad AVL. Parámetros: El elemento a insertar.

- **remove()**

Descripción: Este método permite eliminar un elemento del Árbol AVL. También se encarga de mantener la propiedad AVL al realizar la eliminación. Parámetros: El elemento a eliminar.

- **INPUT**

Una sola palabra en mayúsculas.

- **OUTPUT**

Se debe construir el árbol AVL considerando el valor decimal de su código ASCII.

- **Prueba de las operaciones**

Se deben realizar pruebas exhaustivas de todas las operaciones implementadas para asegurarse de que funcionen correctamente en diferentes escenarios.

- **Estudio de la biblioteca Graph Stream**

Se debe investigar y estudiar la biblioteca Graph Stream para obtener una salida gráfica de la implementación del Árbol AVL. Esto permitirá visualizar de manera más intuitiva la estructura y los cambios realizados en el árbol.

- **Recomendaciones del docente**

Se deben seguir todas las recomendaciones proporcionadas por el docente para garantizar una implementación eficiente y correcta del Árbol AVL. Esto incluye prácticas de codificación, uso adecuado de variables y estructuras de datos, y buenas prácticas de programación en general.

2. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.

- <https://github.com/BastleyNait/EDA-LAB-B-23A.git>

- URL para el laboratorio 04 en el Repositorio GitHub.

- <https://github.com/BastleyNait/EDA-LAB-B-23A/tree/main/lab03>

3. Ejercicios designados

3.1. Clase AVL

- En resumen, este código implementa las operaciones básicas de un Árbol AVL, como inserción, eliminación y búsqueda, junto con funcionalidades auxiliares como obtener el mínimo, máximo y padre de un elemento, realizar recorridos y mantener el equilibrio del árbol.

Listing 1: AVL.java

```
1 import myExceptions.ExceptionNotFound;
2
3 public class AVL<T extends Comparable<T>> {
4     private Nodo<T> root;
5
6     public AVL() {
7         this.root = null;
8     }
9
10    public Nodo<T> getRoot() {
11        return root;
12    }
13
14    public void setRoot(Nodo<T> root) {
15        this.root = root;
16    }
17
18    public boolean isEmpty() {
19        return this.root == null;
20    }
21
22    // Insertar un elemento en el rbol AVL
23    public void insert(T x) throws ExceptionNotFound {
24        this.root = insert(x, this.root);
25    }
26
27    private Nodo<T> insert(T x, Nodo<T> current) {
28        if (current == null) {
29            current = new Nodo<>(x);
30        } else {
31            int bool = current.getData().compareTo(x);
32            if (bool <= 0)
33                current.setRight(insert(x, current.getRight()));
34            else
35                current.setLeft(insert(x, current.getLeft()));
36        }
37        updateHeight(current);
38        return applyRotation(current);
39    }
40
41    // Eliminar un elemento del rbol AVL
42    public void remove(T x) throws ExceptionNotFound {
43        this.root = remove(x, this.root);
44    }
45
46    private Nodo<T> remove(T x, Nodo<T> current) throws ExceptionNotFound {
```

```
47     if (current == null)
48         throw new ExceptionNotFound("No se encontr el elemento");
49
50     int bool = current.getData().compareTo(x);
51     if (bool < 0)
52         current.setRight(remove(x, current.getRight()));
53     else if (bool > 0)
54         current.setLeft(remove(x, current.getLeft()));
55     else {
56         // si tiene un hijo o es hoja
57         if (current.getLeft() == null)
58             return current.getRight();
59         else if (current.getRight() == null)
60             return current.getLeft();
61         // dos hijos
62         current.setData(getMin(current.getRight()));
63         current.setRight(remove(current.getData(), current.getRight()));
64     }
65     updateHeight(current);
66     return applyRotation(current);
67 }
68
69 // Buscar un elemento en el rbol AVL
70 public T search(T x) throws ExceptionNotFound {
71     Nodo<T> aux = search(x, this.root);
72     if (aux == null) {
73         throw new ExceptionNotFound("No se encontr el elemento");
74     }
75     return aux.getData();
76 }
77 //was
78 public Nodo<T> searchNode(T x) throws ExceptionNotFound {
79     Nodo<T> aux = search(x, this.root);
80     if (aux == null) {
81         throw new ExceptionNotFound("No se encontr el elemento");
82     }
83     return aux;
84 }
85
86 private Nodo<T> search(T x, Nodo<T> current) {
87     if (current == null) {
88         return null;
89     } else {
90         int bool = current.getData().compareTo(x);
91         if (bool == 0)
92             return current;
93         if (bool < 0)
94             return search(x, current.getRight());
95         else
96             return search(x, current.getLeft());
97     }
98 }
99
100 // Obtener el elemento mnimo del rbol AVL
101 private T getMin(Nodo<T> node) {
102     T minKey = node.getData();
```

```
103     while (node.getLeft() != null) {
104         minKey = node.getLeft().getData();
105         node = node.getLeft();
106     }
107     return minKey;
108 }
109
110 public T getMin() throws ExceptionNotFound {
111     if (isEmpty())
112         throw new ExceptionNotFound("El rbol est vaco ");
113     Nodo<T> current = root;
114     while (current.getLeft() != null) {
115         current = current.getLeft();
116     }
117     return current.getData();
118 }
119
120 // Obtener el elemento mximo del rbol AVL
121 public T getMax() throws ExceptionNotFound {
122     if (isEmpty())
123         throw new ExceptionNotFound("El rbol est vaco ");
124     Nodo<T> current = root;
125     while (current.getRight() != null) {
126         current = current.getRight();
127     }
128     return current.getData();
129 }
130
131 // Obtener el padre de un elemento en el rbol AVL
132 public T parent(T x) throws ExceptionNotFound {
133     if (findParent(x, this.root) == null)
134         throw new ExceptionNotFound("El nodo no tiene padre");
135     return findParent(x, this.root).getData();
136 }
137
138 private Nodo<T> findParent(T x, Nodo<T> current) {
139     if (current == null || current.getData().equals(x))
140         return null;
141     /*
142      * comparamos si el nodo izq o der del nodo actual es igual al dato que le
143      * pasamos
144      */
145     if (current.getLeft() != null && current.getLeft().getData().equals(x) ||
146         current.getRight() != null && current.getRight().getData().equals(x))
147         return current;
148     /*
149      * Si no es as volvemos a llamar a la funcin hasta que encontremos
150      */
151     int bool = x.compareTo(current.getData());
152     if (bool < 0)
153         return findParent(x, current.getLeft());
154     else
155         return findParent(x, current.getRight());
156 }
157
158 // Recorrido inorden del rbol AVL
```

```
158 public void inOrden() {
159     if (isEmpty())
160         System.out.println("El rbol est vaco ");
161     else
162         inOrden(this.root);
163 }
164
165 private void inOrden(Nodo<T> current) {
166     if (current.getLeft() != null)
167         inOrden(current.getLeft());
168     System.out.println(current);
169     if (current.getRight() != null)
170         inOrden(current.getRight());
171 }
172
173 // Actualizar la altura de un nodo en el rbol AVL
174 public void updateHeight(Nodo<T> current) {
175     int maxHeight = Math.max(
176         height(current.getLeft()),
177         height(current.getRight())
178     );
179     current.setHeight(maxHeight + 1);
180 }
181
182 // Obtener la altura de un nodo en el rbol AVL
183 public int height(Nodo<T> nodo) {
184     return (nodo != null) ? nodo.getHeight() : 0;
185 }
186
187 // Aplicar rotacin segn el balance del nodo en el rbol AVL
188 public Nodo<T> applyRotation(Nodo<T> current) {
189     int balance = balance(current);
190     // Izquierda pesada
191     if (balance > 1) {
192         if (balance(current.getLeft()) < 0) {
193             current.setLeft(rotateLeft(current.getLeft()));
194         }
195         return rotateRight(current);
196     }
197     // Derecha pesada
198     if (balance < -1) {
199         if (balance(current.getRight()) > 0) {
200             current.setRight(rotateRight(current.getRight()));
201         }
202         return rotateLeft(current);
203     }
204     return current;
205 }
206
207 // Realizar rotacin hacia la derecha
208 private Nodo<T> rotateRight(Nodo<T> nodo) {
209     // Para esto tenemos que guardar la informacin de los siguientes nodos:
210     // el nodo izquierdo:
211     Nodo<T> leftNode = nodo.getLeft();
212     // el nodo derecho del nodo izquierdo
213     Nodo<T> centralNode = leftNode.getRight();
```

```
214     leftNode.setRight(nodo);
215     nodo.setLeft(centralNode);
216     updateHeight(nodo);
217     updateHeight(leftNode);
218     return leftNode;
219 }
220
221 // Realizar rotacin hacia la izquierda
222 private Nodo<T> rotateLeft(Nodo<T> nodo) {
223     // Para esto tenemos que guardar la informacin de los siguientes nodos:
224     // el nodo derecho:
225     Nodo<T> rightNode = nodo.getRight();
226     // el nodo izquierdo del nodo derecho
227     Nodo<T> centralNode = rightNode.getLeft();
228     rightNode.setLeft(nodo);
229     nodo.setRight(centralNode);
230     updateHeight(nodo);
231     updateHeight(rightNode);
232     return rightNode;
233 }
234
235 // Calcular el balance de un nodo en el rbol AVL
236 public int balance(Nodo<T> nodo) {
237     return (nodo != null)
238         ? height(nodo.getLeft()) - height(nodo.getRight())
239         : 0;
240 }
241 }
```

- El código implementa una estructura de datos conocida como Árbol AVL.
- Un Árbol AVL es un tipo de árbol binario de búsqueda que garantiza una diferencia máxima de altura de 1 entre sus subárboles izquierdo y derecho.
- La clase AVL es el contenedor principal que representa el Árbol AVL. Está parametrizada con el tipo T, que debe ser un tipo comparable.
- La clase Nodo representa los nodos individuales del Árbol AVL. Cada nodo contiene un dato de tipo T, referencias al nodo izquierdo y derecho, y un campo para almacenar la altura del nodo.
- El método insert permite insertar un nuevo elemento en el Árbol AVL. Se realiza automáticamente el ajuste del árbol para mantener la propiedad AVL.
- El método remove permite eliminar un elemento del Árbol AVL. También se encarga de mantener la propiedad AVL después de la eliminación.
- El método search permite buscar un elemento en el Árbol AVL y devuelve el elemento si se encuentra. Si el elemento no se encuentra, lanza una excepción de tipo ExceptionNotFound.
- El método getMin devuelve el elemento mínimo (menor valor) presente en el Árbol AVL.
- El método getMax devuelve el elemento máximo (mayor valor) presente en el Árbol AVL.
- El método parent devuelve el padre de un elemento dado en el Árbol AVL.
- El método inOrden realiza un recorrido inorden del Árbol AVL e imprime los elementos en orden ascendente.

- Los métodos `updateHeight` y `height` se utilizan para calcular y actualizar la altura de un nodo en el Árbol AVL.
- El método `applyRotation` se encarga de aplicar rotaciones en los nodos del Árbol AVL para mantener la propiedad AVL.
- Los métodos `rotateRight` y `rotateLeft` realizan rotaciones hacia la derecha e izquierda, respectivamente, en los nodos del Árbol AVL.
- El método `balance` calcula el balance de un nodo en el Árbol AVL, que es la diferencia de alturas entre su subárbol izquierdo y derecho.
- La excepción `ExceptionNotFound` se lanza cuando no se encuentra un elemento en el Árbol AVL.
- En resumen, este código implementa las operaciones básicas de un Árbol AVL, como inserción, eliminación y búsqueda, junto con funcionalidades auxiliares como obtener el mínimo, máximo y padre de un elemento, realizar recorridos y mantener el equilibrio del árbol.

3.2. Clase Nodo

- En resumen, esta clase `Nodo` representa un nodo en un árbol binario, utilizado en el contexto del Árbol AVL. Cada nodo contiene un dato, referencias a los nodos hijo izquierdo y derecho, y un campo para almacenar la altura del nodo. Proporciona métodos para acceder y modificar estos campos, así como métodos para obtener representaciones del nodo y realizar diferentes recorridos en profundidad en el árbol.

Listing 2: `Nodo.java`

```
1 public class Nodo <T extends Comparable<T>> {
2     private T data;
3
4     private Nodo <T> left;
5
6     private int height = 1;
7
8     private Nodo <T> right;
9     public Nodo(T data) {
10         this(data,null,null);
11     }
12
13     public Nodo(T data, Nodo <T> left, Nodo <T> right) {
14         this.data = data;
15         this.left = left;
16         this.right = right;
17     }
18     public int getHeight() {
19         return height;
20     }
21
22     public void setHeight(int height) {
23         this.height = height;
24     }
25     public T getData() {
26         return this.data;
27     }
28 }
```

```
29 public void setData(T data) {
30     this.data = data;
31 }
32 public Nodo <T> getLeft() {
33     return this.left;
34 }
35
36 public void setLeft(Nodo <T> left) {
37     this.left = left;
38 }
39
40 public Nodo <T> getRight() {
41     return this.right;
42 }
43
44 public void setRight(Nodo <T> right) {
45     this.right = right;
46 }
47
48 public String toString() {
49     return this.data.toString();
50 }
51
52 //recorridos en profundidad
53 /*
54  * Raiz
55  * Izquierda
56  * Derecha
57  * */
58 // pre-order Rid: 3 1 6 4 5 8 7 9
59 // in-order iRd: 1 3 5 4 6 7 8 9
60 // post-order idR: 1 5 4 7 9 8 6 3
61
62 // busqueda binary/dichotomy
63
64 }
```

- El código define la clase `Nodo`, que representa un nodo individual en un árbol binario, utilizado en el contexto del Árbol AVL.
- La clase `Nodo` está parametrizada con el tipo `T`, que debe ser un tipo comparable.
- El nodo tiene un campo `data` que almacena el elemento o dato contenido en el nodo.
- El nodo tiene una referencia al nodo hijo izquierdo (`left`) y una referencia al nodo hijo derecho (`right`).
- El nodo también tiene un campo `height` que almacena la altura del nodo en el árbol.
- El constructor `Nodo(T data)` crea un nodo con el dato especificado y establece las referencias a los nodos hijo izquierdo y derecho como `null`.
- El constructor `Nodo(T data, Nodo<T> left, Nodo<T> right)` crea un nodo con el dato especificado y establece las referencias a los nodos hijo izquierdo y derecho según los parámetros.
- Los métodos `getHeight` y `setHeight` permiten obtener y establecer la altura del nodo.

- El método `getData` devuelve el dato almacenado en el nodo.
- El método `setData` establece el dato almacenado en el nodo.
- Los métodos `getLeft` y `getRight` devuelven las referencias a los nodos hijo izquierdo y derecho, respectivamente.
- Los métodos `setLeft` y `setRight` establecen las referencias a los nodos hijo izquierdo y derecho, respectivamente.
- El método `toString` devuelve una representación en forma de cadena del dato almacenado en el nodo.
- Los comentarios indican diferentes recorridos en profundidad que se pueden realizar en un árbol (preorden, inorden y postorden), pero no se implementan en el código.
- El comentario "busqueda binary/dichotomy" sugiere que se podría implementar una búsqueda binaria en el árbol, pero no se proporciona la implementación en el código.
- En resumen, esta clase `Nodo` representa un nodo en un árbol binario, utilizado en el contexto del Árbol AVL. Cada nodo contiene un dato, referencias a los nodos hijo izquierdo y derecho, y un campo para almacenar la altura del nodo. Proporciona métodos para acceder y modificar estos campos, así como métodos para obtener representaciones del nodo y realizar diferentes recorridos en profundidad en el árbol.

3.3. Clase Aplicacion

- La clase `Aplicacion` utiliza el árbol AVL implementado en la clase `AVL` para generar y manipular un árbol AVL. Utiliza la biblioteca `GraphStream` para visualizar el árbol en forma de grafo. Los nodos del árbol se agregan al grafo y se establecen las aristas entre los nodos padre e hijo. Luego, se realizan operaciones como eliminar un nodo, mostrar el valor mínimo y máximo, buscar un nodo y buscar los hijos de un nodo. Los resultados se muestran en la consola y se actualiza la visualización del grafo.

Listing 3: `Aplicacion.java`

```
1 import myExceptions.ExceptionNotFound;
2 import org.graphstream.graph.Graph;
3 import org.graphstream.graph.implementations.SingleGraph;
4 import org.graphstream.graph.Node;
5 import org.graphstream.ui.view.Viewer;
6
7 import java.util.Scanner;
8
9 public class Aplicacion {
10     public static void main(String args[]) throws ExceptionNotFound {
11         generar();
12     }
13
14     public static void generar() throws ExceptionNotFound {
15         System.setProperty("org.graphstream.ui", "swing");
16
17         // Crear un grafo
18         Graph graph = new SingleGraph("Aplicacion");
19
20         // Crear un visor para el grafo
```

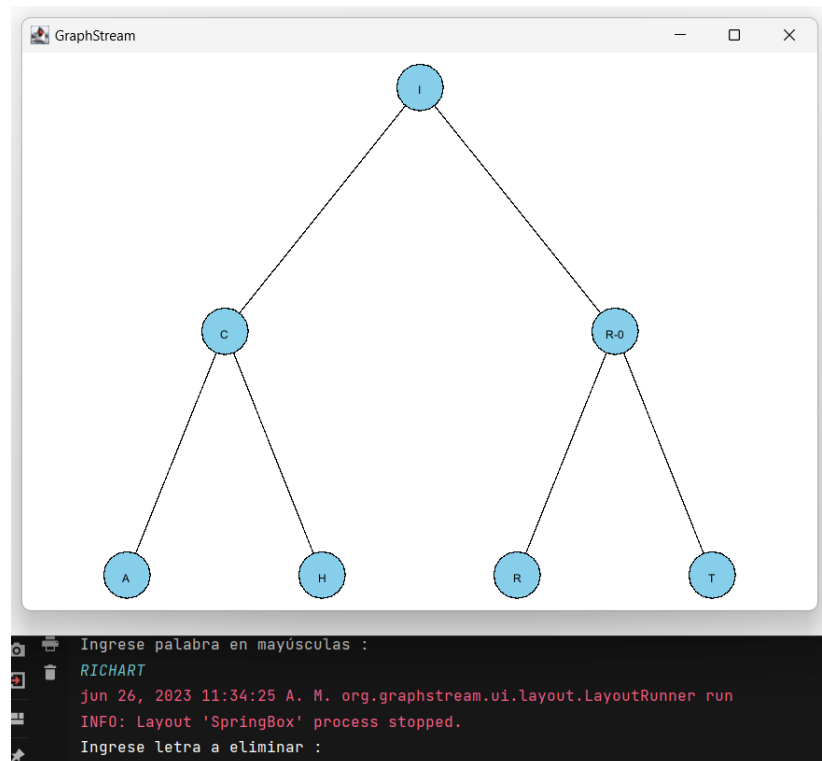
```
21 Viewer viewer = graph.display();
22 viewer.disableAutoLayout();
23
24 // Establecer la hoja de estilo para la representacin visual de los nodos y las
    aristas
25 graph.setAttribute("ui.stylesheet", styleSheet);
26
27 // Leer la entrada del usuario
28 Scanner sc = new Scanner(System.in);
29 System.out.println("Ingrese palabra en mayusculas :");
30 String word = sc.next(); // Ingresa tu palabra en mayusculas aqu
31
32 AVL<String> AVL = new AVL<>();
33 int i = 0;
34
35 char[] words = word.toCharArray();
36 for (char c : words) {
37     try {
38         try {
39             AVL.search(String.valueOf(c));
40             // Si la bsqueda no arroja una excepcin, se ejecuta este bloque
41             AVL.insert(String.valueOf(c) + "-" + i);
42             i++;
43         } catch (ExceptionNotFound e) {
44             AVL.insert((String.valueOf(c)));
45         }
46
47     } catch (ExceptionNotFound e) {
48         e.printStackTrace();
49     }
50 }
51 //
52 // Agregar nodos al grafo
53 addNodesToGraph(AVL.getRoot(), graph, 0.0, 0.0);
54
55 // Crear otro grafo y visor para mostrar el grafo modificado
56 Graph graph2 = new SingleGraph("Aplicacion2");
57 graph2.setAttribute("ui.stylesheet", styleSheet);
58 Viewer viewer2 = graph2.display();
59
60 // Verificar las aristas en el rbol AVL
61 verifyEdges(AVL.getRoot(), graph2);
62 viewer2.disableAutoLayout();
63
64 // Leer la entrada para eliminar un nodo
65 System.out.println("Ingrese letra a eliminar :");
66 String eliminar = sc.next(); // Ingresa tu palabra en mayusculas aqu
67
68 // Eliminar el nodo especificado del rbol AVL y del grafo
69 AVL.remove(eliminar);
70 graph.removeNode(eliminar);
71
72 // Agregar nodos al grafo modificado
73 addNodesToGraph(AVL.getRoot(), graph2, 0.0, 0.0);
74
75 // Mostrar el valor mnimo y mximo en el rbol AVL
```

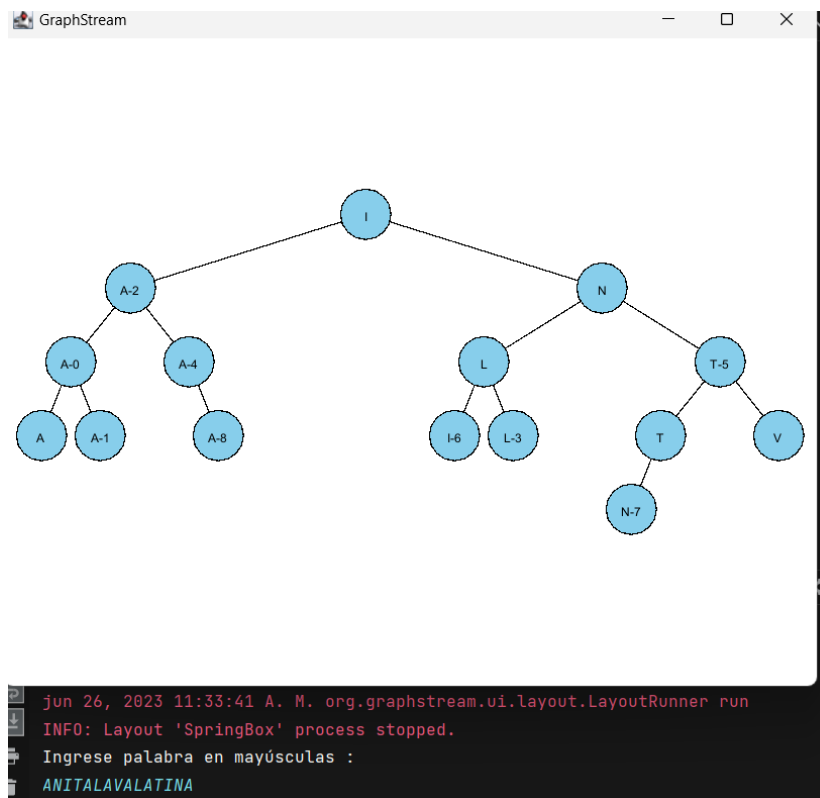
```
76     System.out.println("Valor mnimo: " + AVL.getMin());
77     Node node1 = graph2.addNode(String.valueOf(AVL.getMin()) + "-");
78     node1.setAttribute("ui.label", node1.getId());
79     node1.setAttribute("xy", 2, 2);
80
81     System.out.println("Valor mximo: " + AVL.getMax());
82     Node node2 = graph2.addNode(String.valueOf(AVL.getMax()) + "+");
83     node2.setAttribute("ui.label", node2.getId());
84     node2.setAttribute("xy", 3, 2);
85
86     // Leer la entrada para buscar un nodo
87     System.out.println("Ingrese letra a buscar:");
88     String buscar = sc.next(); // Ingresa tu palabra en mayusculas aqu
89     System.out.println("Buscar " + buscar + ": " + AVL.search(buscar));
90     System.out.println("Padre de " + buscar + ": " + AVL.parent(buscar));
91     Node node3 = graph2.addNode(String.valueOf(AVL.search(buscar)) + "-s");
92     Node node4 = graph2.addNode(String.valueOf(AVL.parent(buscar)) + "-p");
93     node3.setAttribute("ui.label", node3.getId());
94     node3.setAttribute("xy", 4, 2);
95
96     node4.setAttribute("ui.label", node4.getId());
97     node4.setAttribute("xy", 5, 2);
98
99     // Leer la entrada para buscar los hijos de un nodo
100    System.out.println("Ingrese letra a nodo a buscar hijos: ");
101    String son = sc.next(); // Ingresa tu palabra en mayusculas aqu
102}
103
104// Agregar nodos al grafo de forma recursiva
105private static void addNodesToGraph(Nodo current, Graph graph, double x, double y) {
106    if (current != null) {
107        // Agregar el nodo actual al grafo
108        Node node = graph.addNode(String.valueOf(current.getData()));
109        node.setAttribute("ui.label", node.getId());
110        node.setAttribute("xy", x, y);
111        double leftX = x - 10*Math.pow(2,current.getHeight());
112        double rightX = x + 10*Math.pow(2,current.getHeight());
113        double childY = y - 100;
114
115        // Agregar los nodos hijos recursivamente
116        if (current.getLeft() != null) {
117            addNodesToGraph(current.getLeft(), graph, leftX, childY);
118            graph.addEdge((current.getData() + "-" + current.getLeft().getData()),
119                String.valueOf(current.getData()),
120                String.valueOf(current.getLeft().getData()));
121        }
122        if (current.getRight() != null) {
123            addNodesToGraph(current.getRight(), graph, rightX, childY);
124            graph.addEdge((current.getData() + "-" + current.getRight().getData()),
125                String.valueOf(current.getData()),
126                String.valueOf(current.getRight().getData()));
127        }
128    }
129}
130
131// Verificar las aristas en el rbol AVL
```

```
130 public static void verifyEdges(Nodo current, Graph graph) {
131     for (Node nodo : graph) {
132         System.out.println("ID del nodo: " + nodo.getId());
133     }
134 }
135
136 // Hoja de estilo para la representacin visual de nodos y aristas
137 protected static String styleSheet = "node {" +
138     "    shape: circle;" +
139     "    size: 40px;" +
140     "    text-size: 12;" +
141     "    fill-mode: plain;" +
142     "    fill-color: skyblue;" +
143     "    stroke-mode: plain;" +
144     "    stroke-color: black;" +
145     "    stroke-width: 1px;" +
146     "}" +
147     "edge {" +
148     "    arrow-shape: arrow;" +
149     "    arrow-size: 20px, 4px;" +
150     "}" +
151     "node.root {" +
152     "    fill-color: yellow;" +
153     "};
154 }
```

- En el método main, se llama al método generar() para iniciar el proceso de generación del árbol AVL y visualización.
- El método generar() realiza una serie de pasos para generar y manipular el árbol AVL:
 - Establece la propiedad del sistema .org.graphstream.ui.^{en} "swing" para utilizar la interfaz de usuario de Swing para mostrar el grafo.
 - Crea un objeto Graph llamado graph utilizando la implementación SingleGraph.
 - Crea un objeto Viewer para mostrar el grafo y desactiva el diseño automático.
 - Establece la hoja de estilo para la representación visual de los nodos y las aristas del grafo.
 - Lee la entrada del usuario para obtener una palabra en mayúsculas.
 - Crea un objeto AVL<String> llamado AVL para almacenar los caracteres de la palabra y sus índices.
 - Itera sobre los caracteres de la palabra y realiza lo siguiente:
 - Intenta buscar el carácter en el árbol AVL utilizando el método search. Si la búsqueda no arroja una excepción, se ejecuta el bloque try y se inserta el carácter seguido del índice en el árbol AVL utilizando el método insert.
 - Si la búsqueda arroja una excepción ExceptionNotFound, se ejecuta el bloque catch y se inserta solo el carácter en el árbol AVL.
 - Si ocurre una excepción ExceptionNotFound en cualquiera de los bloques try o catch, se imprime la traza de la excepción.
 - Agrega los nodos del árbol AVL al grafo utilizando el método addNodesToGraph, que se explica más adelante.
 - Crea otro objeto Graph llamado graph2 y un Viewer para mostrar el grafo modificado.
 - Verifica las aristas en el árbol AVL utilizando el método verifyEdges.

- Desactiva el diseño automático del Viewer.
 - Lee la entrada del usuario para obtener una letra a eliminar.
 - Elimina el nodo especificado del árbol AVL y del grafo.
 - Agrega los nodos del árbol AVL modificado al grafo graph2.
 - Muestra el valor mínimo y máximo en el árbol AVL y agrega los nodos correspondientes al grafo graph2.
 - Lee la entrada del usuario para buscar una letra en el árbol AVL.
 - Realiza la búsqueda y muestra el resultado, así como el padre del nodo buscado. También agrega los nodos correspondientes al grafo graph2.
 - Lee la entrada del usuario para buscar los hijos de un nodo y continúa con otro código que falta.
- El método addNodesToGraph es un método recursivo que agrega los nodos del árbol AVL al grafo de forma recursiva. Toma el nodo actual, el grafo, las coordenadas x e y para la posición del nodo en el plano. Para cada nodo, se agrega un nodo correspondiente al grafo, se establecen las propiedades y etiquetas necesarias, y se agregan las aristas que conectan los nodos padre e hijo.
 - El método verifyEdges recorre todos los nodos del grafo y muestra el ID de cada nodo.
 - La variable styleSheet contiene una hoja de estilo en formato CSS para la representación visual de los nodos y las aristas en el grafo.
 - Ejecucion del codigo:





3.4. Commits del trabajo

- Acá estan algunos de los commits mas importantes en este trabajo:

```
PowerShell 7.4.0-preview.3
PS C:\Users\Sewas\Desktop\LABS\EDA-LAB-B-23A\lab05> git log
commit 1c21a92cc716ba3486473cad105bcef029d1b056 (HEAD -> main)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jun 26 11:06:06 2023 -0500

    arreglando problemas graficos

commit a68b200641c79aa4a6d592e1c2db30a6029b1a32
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jun 26 11:05:02 2023 -0500

    arreglando problemas graficos

commit d80a54c5ba0ad79a5d01a5dab4a3af6043408b3b (origin/main, origin/HEAD)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 21:47:14 2023 -0500

    modificaciones 4

commit cb0c044ed7cfla1726835ca639a80552e8f8dbbf
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 21:43:43 2023 -0500

    modificaciones 3

commit a612dd4bca519c5af508643799b1aeb68e2bd75b
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 20:56:34 2023 -0500

    mejoras 1
...skipping...
commit 1c21a92cc716ba3486473cad105bcef029d1b056 (HEAD -> main)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jun 26 11:06:06 2023 -0500

    arreglando problemas graficos

commit a68b200641c79aa4a6d592e1c2db30a6029b1a32
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Mon Jun 26 11:05:02 2023 -0500

    arreglando problemas graficos

commit d80a54c5ba0ad79a5d01a5dab4a3af6043408b3b (origin/main, origin/HEAD)
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 21:47:14 2023 -0500

    modificaciones 4

commit cb0c044ed7cfla1726835ca639a80552e8f8dbbf
Author: BastleyNait <schirinosne@unsa.edu.pe>
Date: Thu Jun 22 21:43:43 2023 -0500

    modificaciones 3
```

3.5. Estructura de laboratorio 05

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab05/
| .gitignore
| AVL.java
| Nodo.java
| Aplicacion.java
|
\---latex
```

```
| EDA_lab05_schirinosne.pdf
| EDA_lab05_schirinosne.tex
|
+---img
|     Commits.png
|     ejecucion1.png
|     ejecucion2.png
|     logo_abet.png
|     logo_episunsa.png
|     logo_unsa.jpg
|
\---src
|     AVL.java
|     Nodo.java
|     Aplicacion.java
|
\---myExceptions
|     ExceptionIsEmpty.java
|     ExceptionNotFound.java
```

4. Rúbricas

4.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

4.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumple con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo ítems.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	x	4	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	2	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	2	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	1	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	x	4	
Total		20		19	

5. Referencias

- <https://www.geeksforgeeks.org/introduction-to-avl-tree/>
- <https://www.geeksforgeeks.org/insertion-in-an-avl-tree/>
- <https://www.geeksforgeeks.org/deletion-in-an-avl-tree/>
- <https://docs.oracle.com/javase/tutorial/java/generics/types.html>
- <https://algorithmtutor.com/Data-Structures/Tree/AVL-Trees/>